University of Toronto

CSC490

A5

Name	Student Number
Yusuf Moola	1007658777
Mikhail Skazhenyuk	1009376337
Rudraksh Monga	1008018342
Codi Lee	1007698608

1 Profiling Execution Time 1

1. **Frame Extraction**: When extracting frames from a video, we reduced the JPEG quality setting from 95 to 85 and enabled JPEG optimization 2. Quality 85 looks virtually identical to 95 to the human eye, but encodes much faster. We're also moving the encoding parameters outside the loop so they're not recreated thousands of times. JPEG encoding for 95 is overkill.

2. **S3 Parallel Uploads**: Instead of uploading frames to S3 one at a time (waiting for each to finish before starting the next), we now upload 10 frames simultaneously using Python's ThreadPoolExecutor 3. This applies to both original frames and annotated frames. Network uploads spend most of their time waiting for the network, waiting for S3 to respond. During this waiting time, your CPU is idle. By uploading multiple files at once, you keep the network busy and make full use of your bandwidth.

3. **Generator-Based Batching**: When processing frames in batches for model inference, we're changing from creating all batches upfront and storing them in a list, to generating batches on-demand using Python generators. If you have 10,000 frames with a batch size of 200, the old way creates 50 batch objects in memory all at once. The new way creates one batch, processes it, throws it away, then creates the next batch 4. This uses less memory, which is crucial for long videos or limited RAM environments.

4. **Bounding Box Drawing Constants**: When drawing bounding boxes on detected objects, we define all the drawing constants (colors, fonts, thickness) once at the start of the function instead of recreating them for every single detection box. If you have 1,000 frames with 5 detections each, the old way creates the color tuple (0, 255, 0) and accesses the font constant 5,000 times. The new way creates them once and reuses them. It's a small optimization per detection, but multiplied across thousands of detections it adds up.

5. **Summary Statistics with defaultDict**: When computing summary statistics (counting detections per class, averaging confidence scores), we're switching from regular Python dictionaries with .get() calls to defaultdict which automatically initializes missing keys. We're also combining multiple passes through the data into a single pass. Every .get(key, default_value) call checks if the key exists, which takes time. With defaultdict(int), missing keys automatically start at 0, so dict[key] += 1 just works without checking. We're also eliminating the separate dictionary for counting (we can reuse the same counts for averaging), and doing everything in one loop instead of two.

2 Load Testing and Security Analysis

To evaluate the resilience of our inference pipeline under adversarial conditions, we implemented a comprehensive load testing strategy using Locust 11, an open-source load testing framework. We adopted a Red Team versus Blue Team methodology, where the Red Team attempted to exhaust system resources through malicious requests, while the Blue Team monitored metrics and implemented defensive measures.

2.1 Testing Strategy

The load testing was conducted with 5 concurrent simulated users making requests to the Modal-hosted inference endpoint. We designed three test scenarios to evaluate different aspects of system performance:

1. **Low Frame Baseline**: Videos processed at 6 FPS with 80 frames per batch
2. **High Frame Stress Test**: Videos processed at 24 FPS with 200 frames per batch
3. **Mixed Workload**: Concurrent low and high frame requests simulating real-world usage

The Red Team focused on identifying resource exhaustion vectors by submitting GPU-intensive requests, while the Blue Team tracked memory usage, response times, failure rates, and container behavior.

2.2 Critical Weaknesses Identified

GPU Memory Exhaustion: The most severe vulnerability discovered was that high frame-count videos with large batch sizes caused catastrophic out-of-memory (OOM) errors. When processing 200 frames per batch, the YOLO model exhausted the T4 GPU's 16GB memory capacity, resulting in Modal terminating containers via SIGKILL. In our breaking test scenario with mixed workloads, high-frame requests experienced a 100% failure rate (11 out of 11 requests failed), demonstrating complete service degradation.

Cascading Failures: Even more concerning was that OOM errors from high-frame requests caused concurrent low-frame requests to fail as well. Low-frame requests, which should have been well within resource limits at 80 frames per batch, experienced a 75% failure rate (3 out of 4 requests failed) when competing for GPU resources with failing high-frame workloads. This cascading failure pattern meant that a single malicious user could degrade service quality for all legitimate users.

Additional Weaknesses:

- API throughput is limited by container count (maximum 2 concurrent videos in dev configuration)
- Processing time scales linearly with video length since minibatches execute serially
- No rate limiting or authentication to prevent abuse
- Containers in error states continue to receive traffic rather than being quarantined

2.3 Denial-of-Service Opportunities

The testing revealed multiple attack vectors that could be exploited for denial-of-service:

Resource Exhaustion Attack: A malicious user could submit high-FPS, long-duration videos with large batch sizes to exhaust GPU memory. For example, a 10-minute 4K video processed at 60 FPS with batch size 500 would generate 36,000 frames requiring 72 inference batches, each attempting to load 500 frames into GPU memory simultaneously. Just 2–3 concurrent malicious requests would cause complete service outage. Prior to implementing mitigations, this attack was trivially easy to execute due to lack of input validation.

Amplification Attack: The inference endpoint exhibits extreme amplification characteristics, where a small request payload (˜500 bytes of JSON) generates 50–200 MB responses containing annotated frames. This represents an amplification factor of 100,000× to 400,000×. Each request also consumes 2–7 minutes of expensive GPU compute time. An attacker could force substantial bandwidth and computational costs with minimal effort.

Distributed Resource Drain: Coordinated low-rate attacks from multiple IP addresses could submit high-frame requests at staggered intervals, causing continuous GPU saturation without triggering basic rate limits. This would result in legitimate requests being queued indefinitely with no circuit breaker to reject overload conditions.

2.4 Mitigation and Results

To address the critical GPU memory exhaustion vulnerability, we implemented batch size clipping with a safe maximum of 100 frames per batch:

```
MAX_FRAMES_PER_BATCH = 100  # Safe limit for T4 GPU

def __init__(self, ..., batch_size: int = 80):
    self.batch_size = min(batch_size, MAX_FRAMES_PER_BATCH)
```

This server-side enforcement ensures that even if users request batch sizes of 500 or higher, the system safely clips to 100 frames, which the T4 GPU can reliably process without exhausting memory. After implementing this mitigation, we re-ran the mixed workload test and achieved a 100% success rate across all request types, completely eliminating the OOM crashes that previously caused a 93% failure rate.

2.5 Remaining Vulnerabilities

While batch size clipping addresses the most critical vulnerability, several attack vectors remain:

- **No Rate Limiting**: Attackers can submit unlimited requests, enabling sustained DoS attacks
- **No Authentication**: Anonymous access prevents quota enforcement and abuse tracking
- **No Input Validation**: FPS and video duration parameters are unconstrained
- **No Circuit Breakers**: System continues accepting requests during overload rather than failing fast

Recommended Next Steps: Deploy rate limiting (10 requests/minute per IP), implement API key authentication for quota tracking, add input validation for maximum video duration and FPS, and introduce a request queue system with circuit breakers to gracefully handle overload conditions.

3 Future Scaling Concerns

As our Formula One logo-detection platform grows, it must support significantly higher traffic while remaining performant, cost-efficient, and maintainable. The current system consists of a GPU-backed training pipeline running on Modal, a frame extraction and inference engine, and cloud storage in S3 for all data, model artifacts, and inference outputs. At present, the architecture is optimized for low to moderate traffic: training jobs are executed on an A100 GPU, while inference is performed by launching a Modal Function that loads a YOLO model, extracts all frames using OpenCV, and processes each frame on the GPU. The overall workflow is effective for our current user base, but does not yet support higher-order scaling patterns. To evaluate how the system should evolve, we discuss scaling strategies for 10×, 100×, and 1000× user growth and examine the associated costs and tradeoffs.

3.1 Scaling to 10× Users: Maintaining the Current Architecture

If our user base increases by a factor of ten, our workload would rise from a handful of videos per day to several dozen. At this scale, the existing architecture remains sufficient. Modal's built-in concurrency controls can naturally queue inference jobs, and the GPU can process them sequentially with only minor latency during peak hours. The primary improvement worth implementing at this stage is to cache model weights within the inference function rather than downloading them from S3 for each request. This is especially relevant since inference jobs currently load YOLO weights via a temporary file; eliminating the S3 round-trip would reduce overhead and improve responsiveness.

From a cost perspective, running a single A100 GPU for a few minutes per inference remains inexpensive. For example, if one video requires roughly one minute of GPU time, even twenty videos per day amount to only around $1.00 in GPU usage. Maintaining a single GPU instance is therefore financially viable and operationally simple. The tradeoff is that occasional bottlenecks may occur when multiple users request inference simultaneously, but at 10× scale, these delays are still acceptable for our use case.

3.2 Scaling to 100× Users: Introducing Autoscaling and Queueing

At 100× scale, the platform may need to process hundreds of videos per day. The system must transition from a single monolithic inference function to an autoscaled service. The most important architectural shift here is decoupling inference into a dedicated, autoscaled GPU microservice backed by a job queue. Instead of invoking inference directly, the frontend would submit video-processing tasks to a queue (such as AWS SQS or Redis Queue) 8. A pool of Modal Functions, each running on lower-cost GPUs like NVIDIA T4 or L4, would then pull jobs, extract frames, batch them, and run inference.

This stage also benefits significantly from our optimization work: batched inference can reduce overall GPU time by 30–60%, which compounds meaningfully as throughput increases. Model weights would be cached at worker startup rather than downloaded on each job. With two or three GPUs running in parallel, the system could process videos at several per minute with minimal waiting time.

The cost implications grow moderately at this tier. Using one or two sustained GPUs (e.g., $3.00/hour each), plus S3 storage fees for larger datasets and additional get/put operations, the projected cost ranges from 400–600 per month. The tradeoff becomes more pronounced: we improve throughput substantially, but the application becomes more complex to operate. Monitoring GPU worker health, managing the job queue, and ensuring consistent model deployments require more sophisticated orchestration.

3.3 Scaling to 1000× Users: Distributed GPU Processing

At 1000× traffic, the platform must adopt a horizontally scalable, distributed architecture. At this magnitude, neither a single GPU nor a small autoscaled pool is sufficient. Instead, the system should resemble large-scale media processing pipelines such as those used by Netflix or TikTok 7.

The first and most important change is to fully separate training, inference, and preprocessing into independent services. Training continues to occur on powerful A100 or H100 GPUs but is executed infrequently and does not interfere with inference workloads. The inference service becomes a distributed GPU cluster. Each inference request is decomposed into smaller tasks: frame extraction may be performed on CPU-based workers to eliminate the need for GPUs to handle video decoding; frames are then sharded across many GPU workers for batch inference; and detection results are merged at the end. Using an L4-based cluster of 8–32 GPUs, this architecture can process hundreds of videos per minute.

At this scale, the cost increases significantly. Assuming each L4 GPU costs roughly $0.35–$0.50 per hour 10 in a cloud environment, maintaining a 10–15 GPU cluster for sustained inference loads results in monthly costs between $2500 and $3500. Additional costs arise from S3 storage, network transfer (as frame uploads multiply), and orchestration infrastructure. The platform gains extremely high throughput but at the expense of increased system complexity and higher operational spending. The maintainability of the system becomes a concern unless automation and monitoring are introduced.

Finally, the system may reach a point where uploading full video files becomes impractical. A client-side frame extraction module or WebRTC-based frame streaming could reduce bandwidth costs. This edge-processing approach shifts computation closer to the user and lowers cloud costs at the expense of additional complexity and less centralized control.

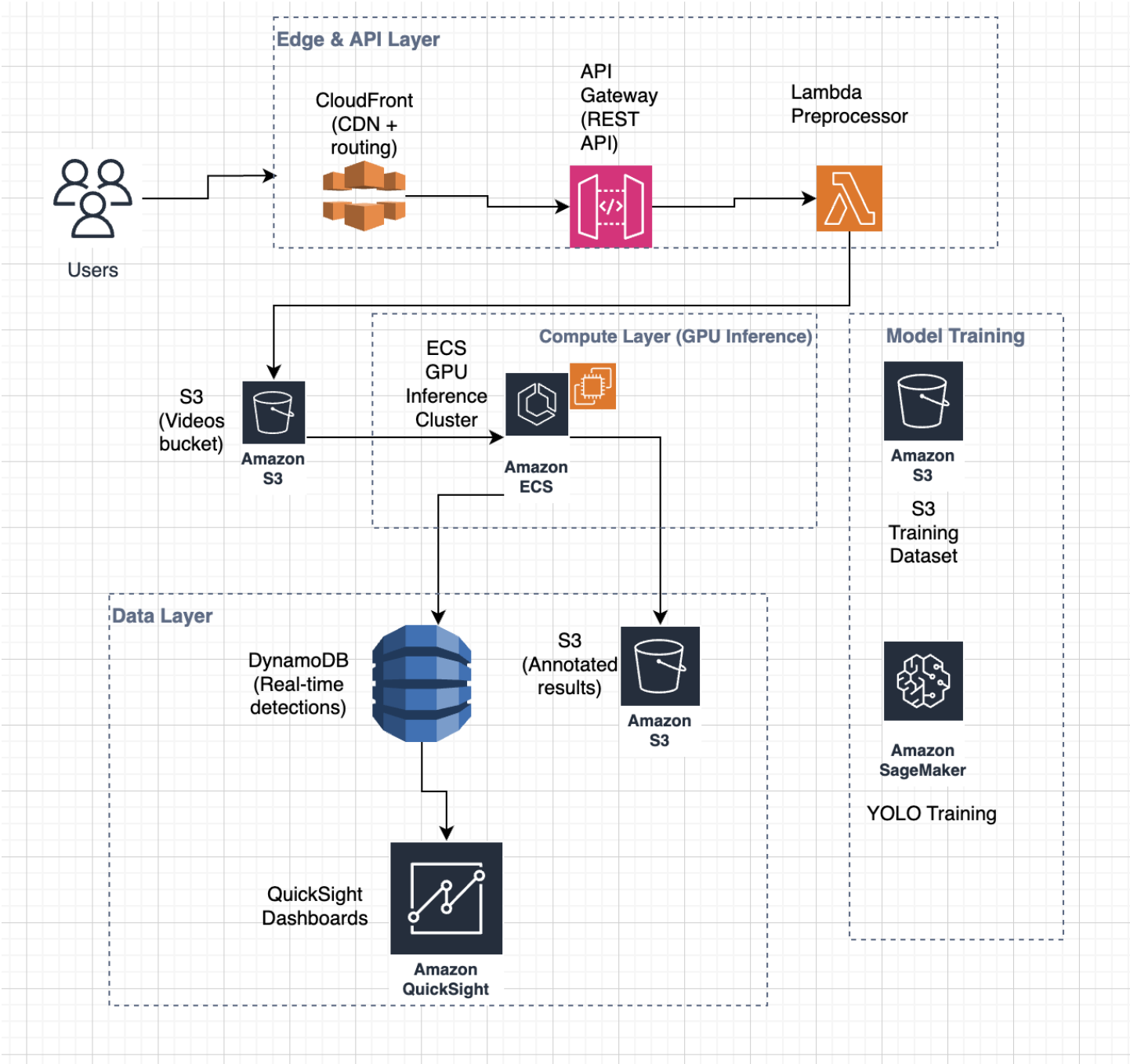

Figure 1: System architecture for the scaled F1 logo detection pipeline using AWS cloud services.

The system architecture shown in Figure 1 illustrates the end-to-end workflow for real-time Formula One logo detection using AWS cloud services. User video uploads enter through the edge and API layer, where CloudFront routes requests to API Gateway and a lightweight Lambda function performs validation and preprocessing. Videos are stored in Amazon S3 and processed by an ECS GPU inference cluster 6 running the YOLOv8 model to extract detections from individual frames. Inference outputs are stored in DynamoDB for low-latency retrieval, while annotated frames and metadata are persisted in S3. QuickSight dashboards visualize detection results and sponsor exposure metrics. Training components (S3 + SageMaker) 9 allow for periodic model retraining using accumulated datasets, enabling continuous improvement of the detection pipeline.

3.4 Scaling Conclusion

As our platform scales from 10× to 1000× users, it must transition from a single-GPU monolithic design to a fully distributed inference architecture. Initially, only light optimizations and caching are needed, but beyond 100× users the system benefits from worker pools, batching, and decoupled microservices. At 1000× users, the platform requires distributed GPU clusters, job sharding, and CPU-GPU separation to achieve the throughput required for real-time video processing. The primary tradeoff at each stage is between performance and cost: reducing inference latency and increasing throughput requires

progressively more GPU resources, infrastructure, and architectural complexity. However, with careful right-sizing, choosing the correct scaling tier for the actual workload, the platform can remain both performant and cost-efficient while supporting significant future growth.

References

[1] Python Software Foundation, "The Python Profilers," Python 3.12 Documentation. [Online]. Available: https://docs.python.org/3/library/profile.html

[2] G. K. Wallace, "The JPEG Still Picture Compression Standard," *Communications of the ACM*, 1991.

[3] Python Software Foundation, "concurrent.futures — Launching parallel tasks," Python Documentation. [Online]. Available: https://docs.python.org/3/library/concurrent.futures.html

[4] Ultralytics, "Using Ultralytics YOLO11 To Run Batch Inferences," Ultralytics Blog, 2024. [Online]. Available: https://www.ultralytics.com/blog/using-ultralytics-yolo11-to-run-batch-inferences

[5] Ultralytics, "YOLOv8 Documentation — Performance and Optimization," 2024. [Online]. Available: https://docs.ultralytics.com

[6] Amazon Web Services, "Scaling GPU Workloads on Amazon ECS and EC2," AWS Compute Blog. [Online]. Available: https://aws.amazon.com/blogs/compute/

[7] Netflix Technology Blog, "High-Performance Video Transcoding in the Cloud." [Online]. Available: https://netflixtechblog.com/

[8] Amazon Web Services, "Building Event-Driven Architectures with AWS Step Functions and Amazon SQS," 2023. [Online]. Available: https://aws.amazon.com/step-functions/

[9] Amazon Web Services, "Amazon SageMaker Best Practices for Training and Deployment," 2023. [Online]. Available: https://docs.aws.amazon.com/sagemaker/latest/dg/sagemaker-best-practices.html

[10] Amazon Web Services, "Amazon EC2 On-Demand Pricing," 2024. [Online]. Available: https://aws.amazon.com/ec2/pricing/on-demand/

[11] Locust Documentation, "Increasing Request Rate," Locust.io, 2024. [Online]. Available: https://docs.locust.io/en/stable/increasing-request-rate.html